Mal: Endpoint-to-Identity Security Architecture

Design rationale and Part 1 retrospective | Arun Muthiah | 28 October 2026 | Build:
https://github.com/marun392/mal-lifecycle-automation

Scope note: Mal is a 50-person fintech scaling to 200, macOS-first, AI-native, operating under UAE financial regulation. Where I had to assume something (fleet, licences, regulator), I say so. Where Part 1 was mocked or unverified, I say that too.

1. Stack design rationale

Principle. Okta is the single policy decision point; every other tool is either a signal source (Jamf, CrowdStrike) or an enforcement point (GWS, 1Password, the apps behind SSO). Nothing gets a login that is not an Okta login, and no Okta login succeeds without a verified user *and* a verified device.

Component	Role in the architecture	Why this choice
Okta	IdP, SSO (SAML/OIDC) for everything, group-based assignment, SCIM to GWS and 1Password, device trust via Okta Verify / FastPass, policy engine.	Already the control plane in Part 1. Groups sourced from HRIS so access follows role, not tickets.
Jamf Pro	MDM for the Mac fleet: enforces FileVault, firewall, screen lock, OS currency; installs Okta Verify and the Falcon sensor; is the authoritative source of 'this device is ours'.	Mac-native; Jamf-to-Okta integration registers devices for Device Trust. Intune only if a Windows fleet appears.
CrowdStrike Falcon (EDR)	Prevention plus detection on every endpoint; Zero Trust Assessment (ZTA) score per device; host containment.	Chose Falcon over Defender/Harmony because its ZTA score is consumed natively by Okta policies and its macOS sensor is mature. Defender would be the pick on a Windows/M365 estate.
Google Workspace	Mail, Drive, Docs. SAML-only users (no Google passwords), SCIM-provisioned, Context-Aware Access as a second device gate, DLP and OAuth app controls.	Second enforcement point so a stolen Google session cannot bypass Okta device checks.
1Password Business	Human secrets. Unlock with Okta (SSO), SCIM bridge for group membership, Events API to SIEM.	Vault access is group-based, so Okta group changes cascade; no standalone 1Password passwords exist.

Trust boundaries

- **Identity boundary.** An Okta session is the only trust anchor. Phishing-resistant MFA (FastPass/WebAuthn) for all apps; SMS and voice disabled; push-only MFA permitted nowhere.
- **Device boundary.** *Managed* means Jamf-enrolled, Okta Verify registered with hardware-bound key, Falcon sensor reporting. Unmanaged devices get Okta self-service and nothing else on day one; a browser-only allowlist (Slack, Gmail read) is a later concession, not a default.
- **Privilege boundary.** Okta, GWS, Jamf, Falcon and AWS admin roles live on separate admin accounts, require FastPass on a managed device with ZTA at or above 80, no open Falcon detection, and a one-hour session. Standing admin membership is reviewed monthly.
- **Automation boundary.** Service identities (the Part 1 script, SCIM, CI) are distinct from humans, scoped per integration, injected from a secrets manager, and never hold admin on more than one system.
- **Data boundary.** Customer PII and financial records live in GWS and production systems behind Context-Aware Access and DLP (section 2); nothing is reachable from an unmanaged device.

How device posture feeds conditional access

Three signals arrive in Okta: Jamf compliance (smart groups synced to Okta device attributes: encryption, OS version, firewall, last check-in), Okta Verify's own attestation (managed, secure hardware, screen lock), and the Falcon ZTA score (0 to 100, derived from OS settings and sensor health) plus containment state. Okta authentication policies combine them per app tier:

Tier	Examples	Required posture	Session
0 - low	Slack, Confluence read, Okta self-service	Registered device, any phishing-resistant factor	7 days
1 - standard	GWS, 1Password, GitHub, Jira, HR portal	Managed device + FastPass + ZTA at or above 60 + FileVault on	12 hours, re-auth on new IP country
2 - privileged	Okta/GWS/Jamf admin, AWS production, banking partner portals	Managed + hardware-bound FastPass + ZTA at or above 80 + no open Falcon detection + Falcon not contained	1 hour, step-up every access
Deny	Any tier	Falcon host contained, FileVault off, OS older than N-1 major, device not seen by Jamf for 14 days	n/a

Failure modes are a decision, not an accident. If the Falcon-to-Okta signal is missing (sensor outage, API failure) the device is treated as *unknown*: tier 1 still opens with a step-up factor, tier 2 fails closed. If Jamf is unreachable, Okta uses its last-synced device attributes for up to 24 hours. Falcon containment is the one signal that denies immediately

regardless of tier, because it is the only one that means an active compromise rather than drift.

2. DLP and email security posture

At 50 people the realistic exfiltration paths are email to personal addresses, Drive links, OAuth-connected apps (including AI assistants), USB, and paste into browser tools. I would cover those five on day one and accept residual risk from determined insiders until classification exists. The data that matters: KYC documents, customer PII, transaction records, credentials, source code.

Layer	Day one (weeks 1 to 4)	Deferred, and why
Email (GWS)	Gmail DLP rules on predefined detectors (card numbers, IBAN, Emirates ID, passport) in *audit* mode, flipped to *block for external recipients* after two weeks of false-positive review. Auto-forwarding to external addresses disabled. Enhanced pre-delivery scanning and attachment sandboxing on. External-sender banner. Confidential mode not relied upon.	Content-aware blocking of attachments by volume (e.g. spreadsheets with more than N PII rows) until Drive labels exist, otherwise the rule is noise. S/MIME gateways: TLS plus access controls are enough for this size.
Endpoint (Jamf, Falcon)	FileVault enforced; USB mass storage read-only by configuration profile; AirDrop receiving-only; Chrome managed via GWS with upload blocking to non-allowlisted domains and paste controls for generative-AI sites; Falcon prevention policies at vendor-recommended aggressive level for macOS.	Falcon Data Protection or a dedicated endpoint DLP agent until data classification is adopted (around 120 people). Screenshot or clipboard blocking: high friction, low value at this size.
SaaS and OAuth	No SaaS without Okta SSO. GWS API controls set to *trusted apps only*: any third-party app requesting Drive or Gmail scopes must be allowlisted. Drive sharing defaults internal; external sharing only to allowlisted partner domains with a warning. This is the same OAuth token surface that Part 1 revokes on offboarding.	CASB (Netskope or Zscaler) until unsanctioned SaaS cannot be managed through the Okta app catalogue and GWS token controls, realistically past 120 people. Drive labels and IRM at 60 to 90 days once a four-level classification is agreed.

Email authentication

SPF: `v=spf1 include:_spf.google.com -all` with a hard fail, keeping the lookup count under ten by using subdomains for bulk senders. **DKIM:** 2048-bit keys for Google and a separate selector per third-party sender (HRIS, CRM, support desk) so one leaked key is one revoked selector. **DMARC:** start at `p=none` with aggregate reports for two to four weeks to find every legitimate sender, move to `p=quarantine; pct=100`, and reach `p=reject; sp=reject` within 90 days. Relaxed alignment initially, because strict alignment breaks forwarded mail before it stops any attacker. Why enforce fully: a fintech's domain is the lure in customer-facing phishing, and the regulator and partner banks will ask for the DMARC record. Also on day 30: MTA-STS and TLS-RPT, and registration of obvious look-alike domains. Inbound: Gmail enforces DMARC on receipt; I would add a quarantine rule for messages that fail DMARC from domains we exchange money with.

Scaling 50 to 200. The shift is from allowlists to labels: Drive labels with automated classification drive sharing rules and DLP, a SIEM ingests GWS audit, Okta system log, Falcon detections and 1Password Events, and the DLP incident queue gets an owner outside IT.

3. What I cut in Part 1 and why

Part 1 is a Python CLI that runs joiner and leaver steps across Okta, GWS, 1Password, Jamf and Slack through one HTTP transport abstraction, so the same client code runs against live APIs or in-process stateful mocks. 47 unit tests, ruff-clean, CI on three Python versions.

Integration	What was built	Status
Okta	Create and activate user, group lookup and assignment, revoke sessions, suspend or deactivate; idempotent on duplicate login.	**Verified live** against an Integrator Free Plan org, both workflows, evidence in docs/VERIFICATION.md
Slack alert	Block Kit message: user, timestamp, duration, every step with status, failures section, live/mock footer.	**Verified live** via incoming webhook
Google Workspace	Create user with forced password change, list and delete every OAuth token, sign out, suspend. Service-account auth with domain-wide delegation.	Implemented against the documented API; **not run live** (no Workspace trial)
1Password	SCIM bridge for user and group membership (remove from every group, then suspend), Connect for vault lookup, Events API for audit confirmation.	Implemented; **not run live** (needs Business tenant with SCIM bridge)
Jamf Pro	Bearer token exchange, XML computer invitation, read-back of the invitation; payload and response logged.	**Mocked by design**, as the brief permitted

Shortcuts taken, stated plainly

- **No retries or rate-limit handling.** Okta returns 429 with a reset header; the script treats it as a failure and moves on. Correct for offboarding (maximal revocation beats tidiness) but a production run needs bounded backoff.

- **Okta SSWS token instead of an OAuth service app.** Faster to set up; in production the automation gets a service app scoped to `okta.users.manage` and `okta.groups.manage`.
- **Secrets from a .env file.** Right interface (environment variables, never logged), wrong store. Production injects from AWS Secrets Manager or a 1Password service account at run time.
- **No ticket or approver in the run record, no HRIS trigger, no approval gate for deactivate.** The tool executes; the decision is still a human typing a command.
- **Mocks are in-memory per process.** Offboarding a user onboarded in an earlier run returned 404 from the mock Google and 1Password tenants. The log records this honestly; it also demonstrated that continue-on-failure and skip semantics work.
- **GWS user gets a generated temporary password.** At Mal, GWS users would be SAML-only with a random password that is never communicated.
- **Coverage gaps.** No AWS, GitHub, LLM-provider API keys, Google application-specific passwords, or device lock and wipe through Jamf on departure. No mover workflow.
- **One incident worth owning.** GitHub push protection blocked the first push because the mock Slack webhook URL matched the real pattern. I replaced it with a non-matching placeholder rather than overriding the block.

What changes in production at Mal. Trigger from the HRIS termination event with the ticket ID carried through to the alert; approval gate before deactivate; Okta group removal doing most of the work through SCIM so direct SaaS calls shrink to the few systems without SCIM; Jamf remote lock on departure and erase after the retention window; Falcon containment when the departure is adverse; run records shipped to the SIEM; retries, a concurrency lock, and a dedicated runner with no human login.

4. Compliance mapping

I map to ISO/IEC 27001:2022 because its control wording is public and precise. CBUAE technology-risk expectations for licensed entities cover the same joiner-mover-leaver ground; I have not quoted clause numbers because I could not verify the current text and would rather confirm it against Mal's licence than guess.

Control	How the Part 1 automation satisfies it	Gaps needing extra audit evidence
A.5.16 Identity management - the full life cycle of identities is managed.	One identity per person, created in Okta and propagated to GWS and 1Password from the same employee record; creates are idempotent so re-runs cannot spawn duplicates; leaver identities are suspended, not deleted, preserving attribution. Each run emits a timestamped JSON record of every identity action with the vendor-side identifier returned.	No evidence of *periodic* reconciliation between HRIS and Okta (orphan or dormant accounts). No link from the run record to an approved request. Run records are written to disk, not an immutable store. Service identities used by the tool are not themselves inventoried.
A.5.18 Access rights - access rights are provisioned, reviewed, modified and removed per policy.	Provisioning is group-based (role, not person), so rights are consistent and reviewable. Removal is complete and ordered: Okta sessions revoked before suspension, Google OAuth tokens revoked individually, 1Password membership removed from every group. Failures cannot be silent: the alert lists them and the exit code is non-zero.	The *review* half of the control is absent: no scheduled access recertification output. Timeliness cannot be proven without the HR termination timestamp to measure against. Coverage is five systems; an auditor will ask for AWS, GitHub and any system without SSO. Live verification exists for Okta and Slack only; GWS and 1Password need a recorded run in Mal's tenants.

Net position. The automation turns two controls from a checklist into repeatable, evidenced actions, and its most useful property for an audit is that it refuses to hide failure. The remaining evidence is process evidence, not tooling: approvals, reviews, immutable retention, and a verified run in each real tenant.